Caja Negra — Doc de diseño (1 página)

Problema. Un servicio HTTP sin código ni documentación (`POST /msg` con `"CMD;args;CONTROL"`, un mini-libro contable con cuentas, depósitos, transferencias y anulaciones) debe reemplazarse respondiendo **exactamente igual (status + cuerpo), errores incluidos**, en 100 tests ocultos que empiezan con `/__reset`. Dos productos: **conocimiento** (log + spec con evidencia) y **reemplazo** desplegado. El original cambia a las 15:30.

Supuestos, casos difíciles y fuera de alcance

Supuestos (por qué)	Casos difíciles (cómo)	Fuera de alcance (por qué)
Determinista tras reset: IDs `AC-0001`/`OP-0001` secuenciales (ejemplos) · El control depende solo del texto (mismo mensaje → mismo control) · Un estado basta: el set usa el token del equipo; otro token → `401 E001` · Lo no observado recibe un default marcado **supuesto** (regla del reto)	**Valor de control** (mod 97, ISO 7064): ajuste + validación sobre mensajes no vistos · **Precedencia de errores**: mensajes con 2 defectos a la vez · **Estado**: secuencias (alta → depósito → transfer → anula → consulta) · **Rarezas** a preservar: `"recieved"`, `cuentaId`/`cuenta_id`, espacio final en E301 · **Rutas/métodos** desconocidos y ruta antes de auth · **Cambio 15:30**	Latencia y headers (`X-Req`): el jurado no los compara · LLM en runtime: el servicio es determinista; añade latencia y ruido · Persistencia durable: el estado se resetea antes de cada test · Bugs de concurrencia: no observables con fiabilidad en un día

Arquitectura — ver diagrama-arquitectura.png

A · Descubrimiento (único que habla con el original) → **Conocimiento** (`experimentos.jsonl` → ajuste → spec; corpus dev/hold-out) → **B · Reemplazo hexagonal**: adaptador HTTP ASGI catch-all → **pipeline ordenado** *ruta → auth → forma del cuerpo → control → sintaxis/aridad → existencia → negocio* (familias de error 0xx/2xx/3xx/4xx/5xx) guiado por un **catálogo de errores en datos** (código, status, cuerpo exacto, etapa) → **registro de comandos** (un módulo por ALTA, CONSULTA, DEPOSITO, TRANSFER, ANULA, PING) → **núcleo de dominio puro** `(estado, cmd) → (estado', resp)` → **puerto StateStore** (`InMemoryStore` hoy; `RedisStore` posible) + generadores de IDs deterministas.

Escalabilidad, con honestidad: la capa de aplicación no guarda estado propio (todo pasa por el puerto), así que escala horizontalmente con un store compartido y partición por token. Desplegamos **1 réplica a propósito**: el set habla a un solo proceso y varias réplicas en memoria partirían el estado.

Decisiones clave

Decisión	Elegimos	Descartamos	Por qué
Núcleo HTTP	ASGI puro catch-all	FastAPI/Flask routing	Responden solos 404/405/422 y redirigen `/x/`: no-paridad justo donde el set prueba
Estructura	Hexagonal: pipeline + registro + dominio puro	Un handler monolítico con `if`	Cada regla nueva toca un módulo; el dominio se prueba sin HTTP
Errores	Catálogo de datos + orden explícito de etapas	Excepciones dispersas en código	La precedencia es el caso difícil; en datos se compara contra el log y se versiona
Estado	Memoria tras puerto, 1 proceso/1 réplica	Redis/Postgres; varias réplicas	Reset antes de cada test; "el set habla a un proceso": N réplicas = N estados. Escalar = Redis + partición por token, sin tocar el dominio
Cambio de versión	`BEHAVIOR_PROFILE=v1|v2` (base + overrides)	Editar reglas in situ / ramas	Ambas versiones conviven y se testean; rollback = 1 variable
Descubrimiento	Prober con log + ajuste + validación predictiva	`curl` manual; herramientas auto	Sin rastro no hay evidencia ni regresión; un ajuste que no predice memoriza
Confianza en spec	`confirmado` = repetido ≥2 o predicho	Visto una vez = confirmado	Las afirmaciones falsas restan

Cómo absorbe el cambio (15:30)

1) Re-ejecutar el corpus dev contra el original → **diff por comportamiento** contra el corpus de la mañana. 2) Cada diferencia se vuelve una fila de spec (§6) y un **override** en `profiles/v2` (catálogo o módulo de comando), sin tocar el núcleo. 3) Desplegar con `BEHAVIOR_PROFILE=v2`; el tester verifica v2 contra el corpus nuevo y v1 contra el viejo. Lo que no se pudo reproducir queda **supuesto**.

Cómo medimos

Métrica (comparador idéntico al jurado)	Objetivo
Tests oficiales (`/evento/resultados`)	≥ 90/100
Paridad de casos en **hold-out** (todas las respuestas iguales) / ponderada 40-45-15 como el set	≥ 90 %
Paridad de pasos / de status	≥ 97 % / ≥ 99 %
Brecha dev − hold-out (sobreajuste)	< 5 pts
Spec: filas con id de log / `confirmado` que cumplen la regla	100 % / 100 %
Presupuesto (8.000 req/día/token) · latencia p95	< 6.000 antes de 18:00 · < 1 s

Caso concreto (sale del diseño)	Request (tras reset)	Esperado
Precedencia ruta > auth	`GET /nada` sin token	`404 E404 ruta desconocida`
Precedencia control > aridad	`ALTA` con 2 args y control malo	`400 E202`, no `E302`
Límite de negocio	`TRANSFER` 100000 vs 120000	`200` vs `422 E506 límite excedido`
Rareza a preservar	2× `ANULA` de la misma op	2.ª → `200` con `ok:false` (no error)
Estado	alta → depósito → transfer → consulta	saldos e IDs `OP-000n` exactos

Categorías (reset + N requests, mezcla 40 % de 1 / 45 % de 2–5 / 15 % de 6–12): **normales** (cada comando válido, secuencias con estado) · **límite** (montos 0, negativos, decimales, 100000 vs 120000, saldo exacto, nombres largos/unicode, mayúsculas `CONSULTa`) · **adversariales** (control ±1 y no numérico, 2 defectos a la vez para precedencia, rutas/métodos/slash, auth en minúsculas o ajena, cuerpo no JSON o sin `m`, ANULA doble). Un comando: `python -m bench.diff --target URL --split holdout`, desglosado por categoría.